Experiment 5.1.1: Leap Year Checker

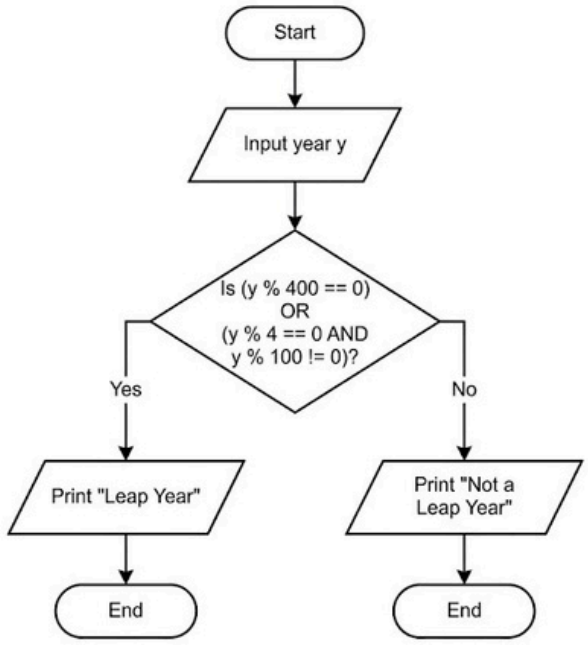
1. Aim

Todesignandimplement a Python program that prompts the user to enter a year and determines whether the given year is a leap year or not, printing the appropriate message.

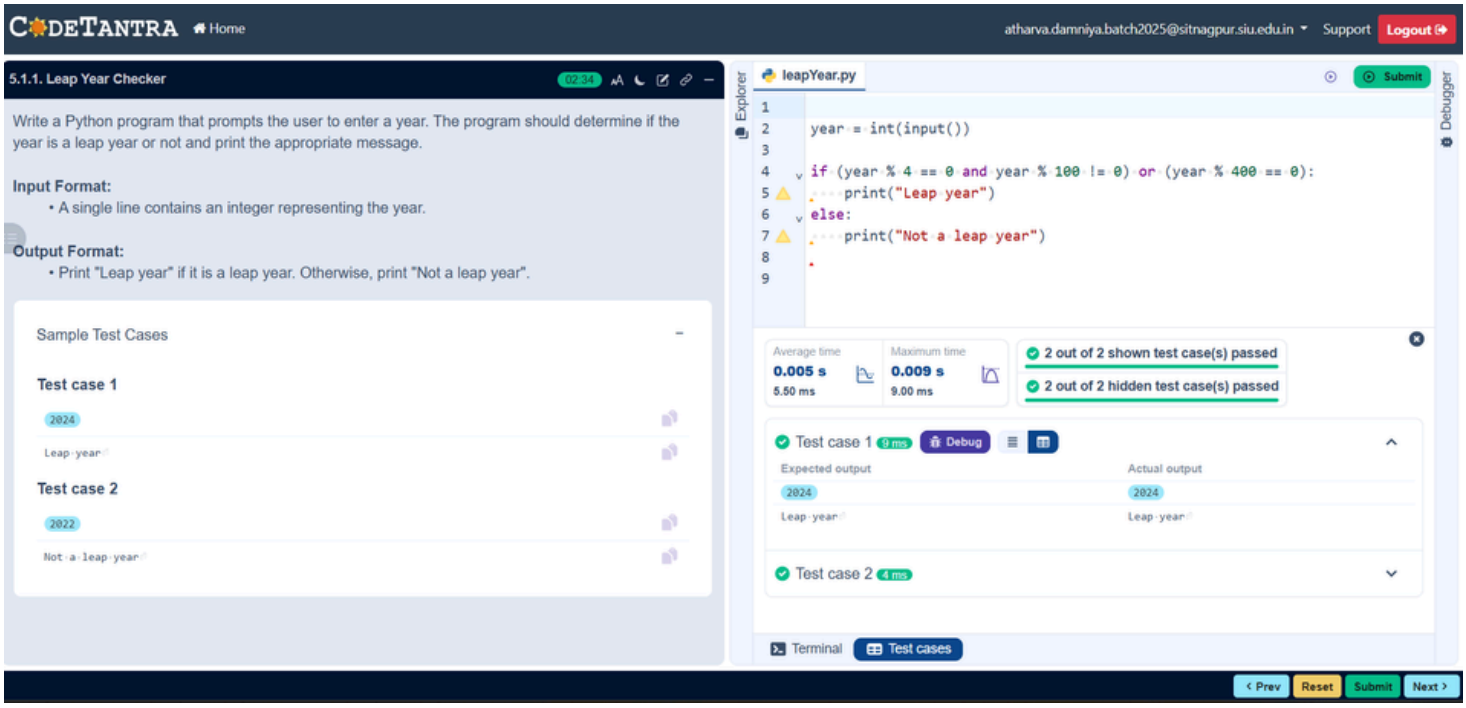
2. Pseudocode

1. **START**
2. **READ** the input value from the user and convert it to an integer.
3. **STORE** this value in the variable y.
4. **CHECK** if the year y is divisible by 400 (y % 400 == 0) **OR** check if the year y is divisible by 4 **AND** not divisible by 100 (y % 4 == 0 and y % 100 != 0).
5. **IF** the condition in step 4 is True, **PRINT** "Leap year".
6. **ELSE**, **PRINT** "Not a leap year".
7. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 5.1.2: Student Grade Based on Aggregate

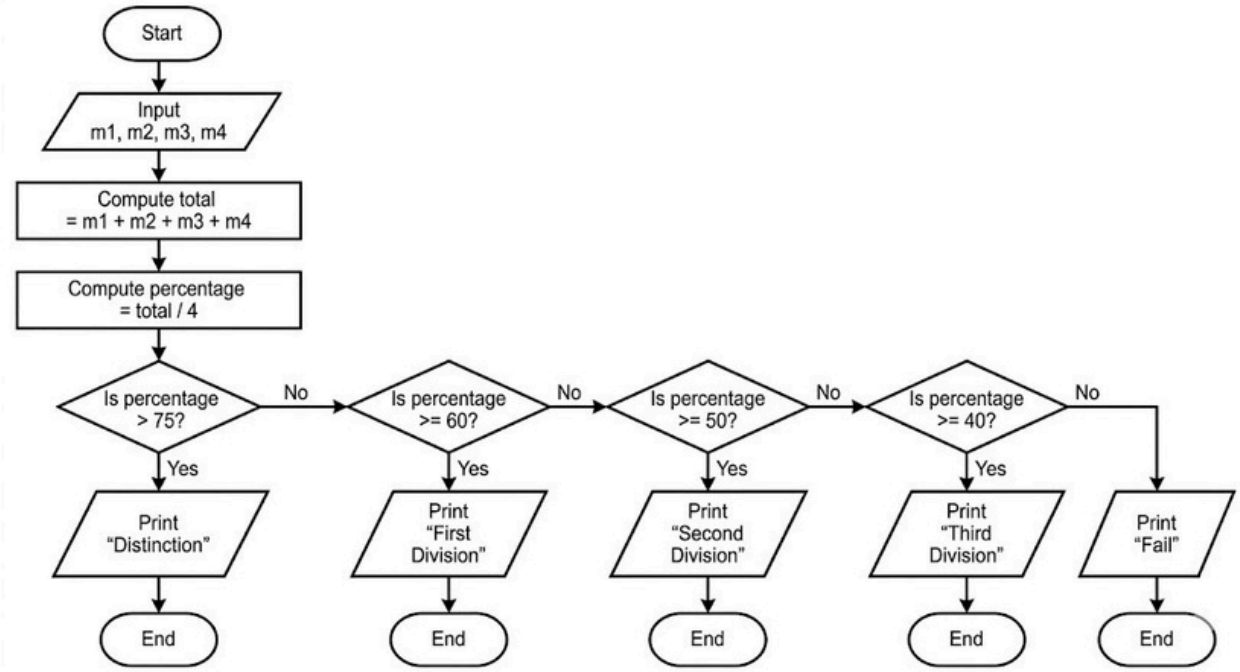
1. Aim

Todesignandimplement a Python program that calculates a student's total marks, aggregate percentage, and final grade based on the marks obtained in four subjects. The program determines the grade using specific percentage-based conditions.

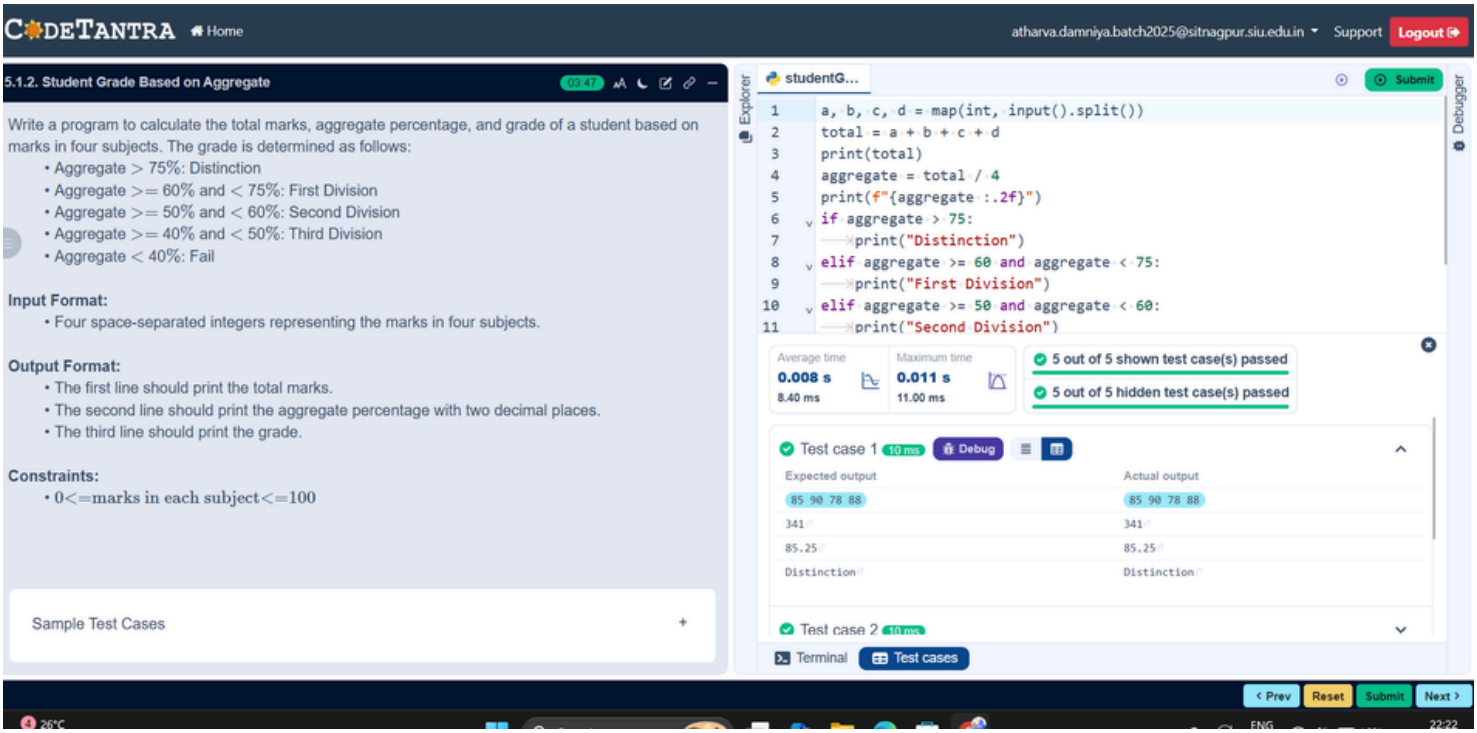
2. Pseudocode

1. **START**
2. **READ** four space-separated integers from the user and **STORE** them in variables m1, m2, m3, and m4.
3. **CALCULATE** the sum of the four subjects and **STORE** it in the variable total ($total = m1 + m2 + m3 + m4$).
4. **CALCULATE** the aggregate percentage by dividing the total by 4 and **STORE** it in the variable percentage.
5. **CHECK** the value of percentage to determine the grade:
   - **IF** percentage > 75, **STORE** "Distinction" in grade.
   - **ELSE IF** percentage >= 60, **STORE** "First Division" in grade.
   - **ELSE IF** percentage >= 50, **STORE** "Second Division" in grade.
   - **ELSE IF** percentage >= 40, **STORE** "Third Division" in grade.
   - **ELSE**, **STORE** "Fail" in grade.
6. **PRINT** the total marks.
7. **PRINT** the percentage formatted to two decimal places.
8. **PRINT** the grade.
9. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 6.1.1: Incremented Date

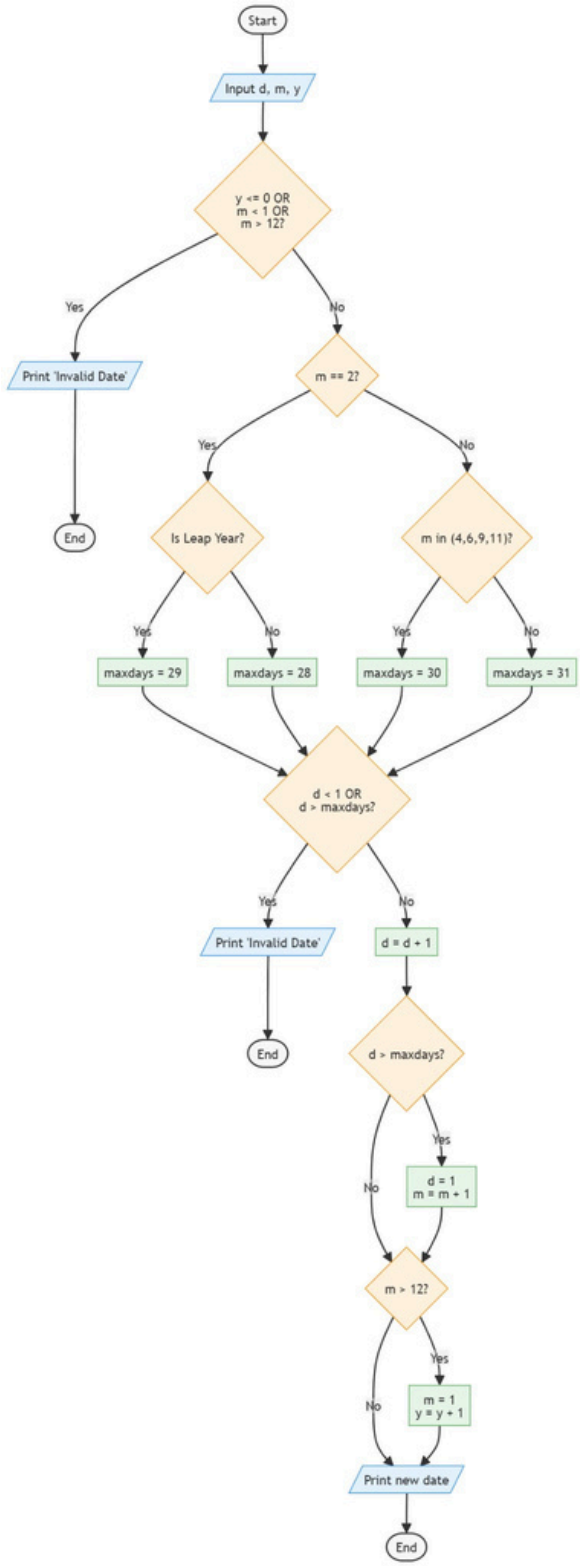
1. Aim

Todesign and implement a Python program that checks if a given date (day, month, year) is valid according to the Gregorian calendar rules. If the date is valid, the program computes and prints the next day's date (incremented date) in DD-MM-YYYY format. If the date is invalid, it prints "Invalid Date".

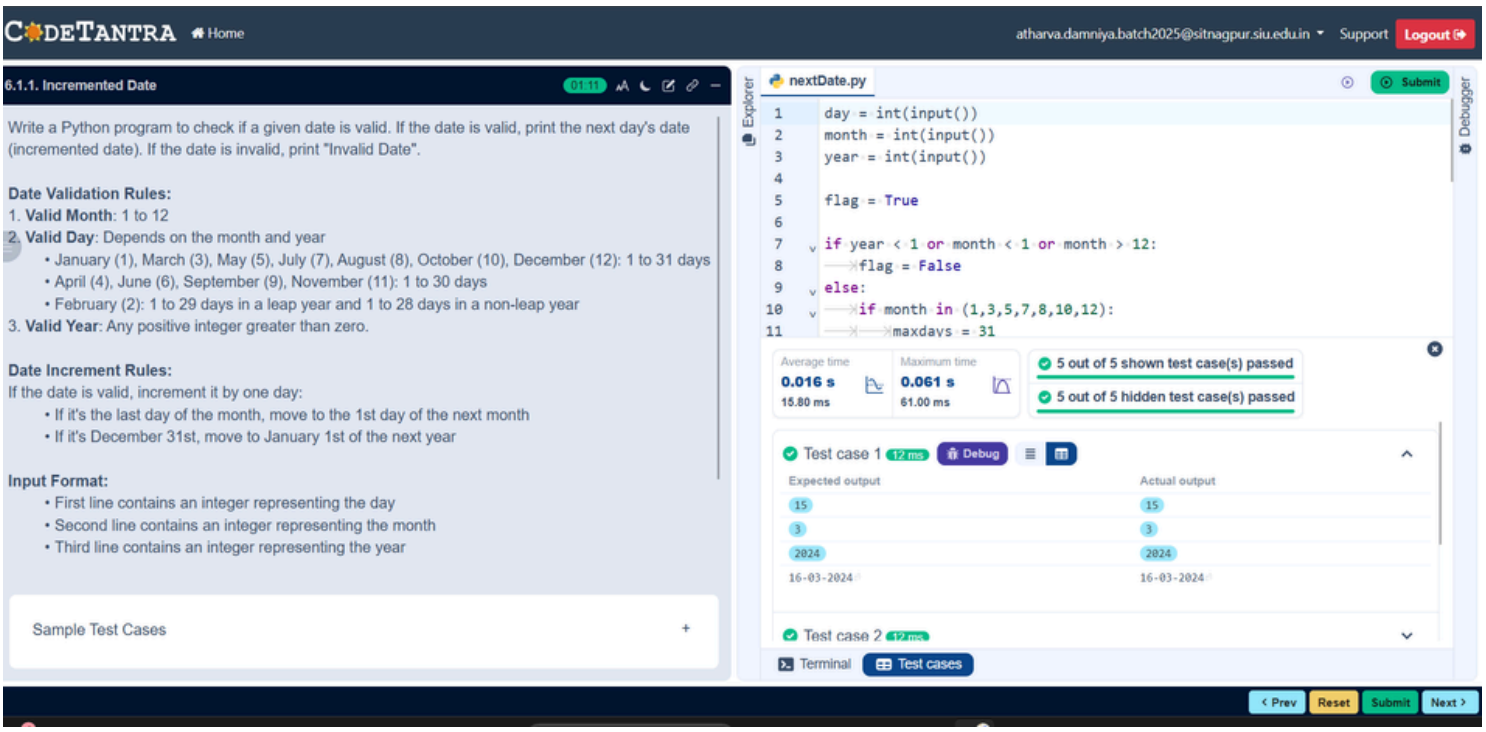
2. Pseudocode

1. **START**
2. **READ** three integers representing the day, month, and year, and **STORE** them in variables d, m, and y.
3. **INITIALIZE** a boolean variable flag to True.
4. **CHECK** basic validity: **IF** $y \leq 0$ **OR** $m < 1$ **OR** $m > 12$, **SET** flag to False.
5. **IF** flag is True:
   - **DETERMINE** maxdays based on the month:
     - **IF** m is 1, 3, 5, 7, 8, 10, or 12: maxdays = 31.
     - **ELSE IF** m is 4, 6, 9, or 11: maxdays = 30.
     - **ELSE IF** m is 2 (February):
       - **IF** the year y is a leap year: maxdays = 29.
       - **ELSE**: maxdays = 28.
   - **CHECK** day validity: **IF** $d < 1$ **OR** $d > maxdays$, **SET** flag to False.
6. **IF** flag is True:
   - **INCREMENT** the day d by 1.
   - **IF** d > maxdays (End of month):
     - **SET** d = 1.
     - **INCREMENT** the month m by 1.
     - **IF** m > 12 (End of year):
       - **SET** m = 1.
       - **INCREMENT** the year y by 1.
   - **PRINT** the incremented date formatted as DD-MM-YYYY (zero-padded).
7. **ELSE**:
   - **PRINT** "Invalid Date".
8. **END**

3. Flowchart


Start
Input d, m, y
y <= 0 OR
m < 1 OR
m > 12?
Yes
No
Print 'Invalid Date'
End
m == 2?
Yes
No
Is Leap Year?
m in (4,6,9,11)?
Yes
No
Yes
No
maxdays = 29
maxdays = 28
maxdays = 30
maxdays = 31
d < 1 OR
d > maxdays?
Yes
No
Print 'Invalid Date'
End
d = d + 1
d > maxdays?
Yes
No
d = 1
m = m + 1
m > 12?
Yes
No
m = 1
y = y + 1
Print new date
End

4. Experiment Screenshot



Experiment 6.1.2: Factorial of a Number

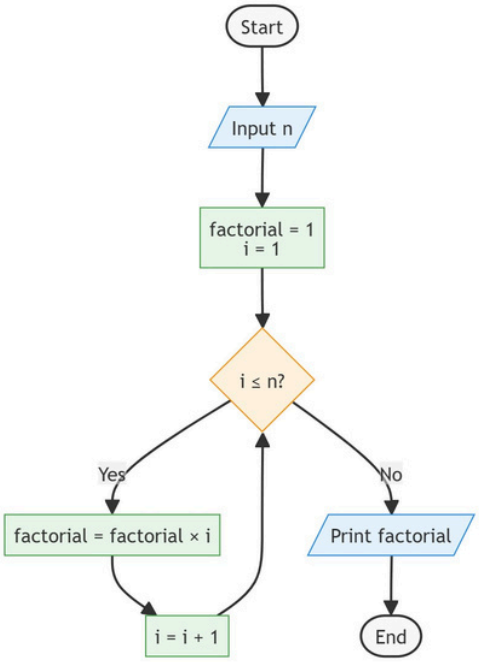
1. Aim

To design and implement a Python program that calculates the factorial of a given integer using a for loop. The program reads the integer, computes its factorial iteratively, and prints the final result.

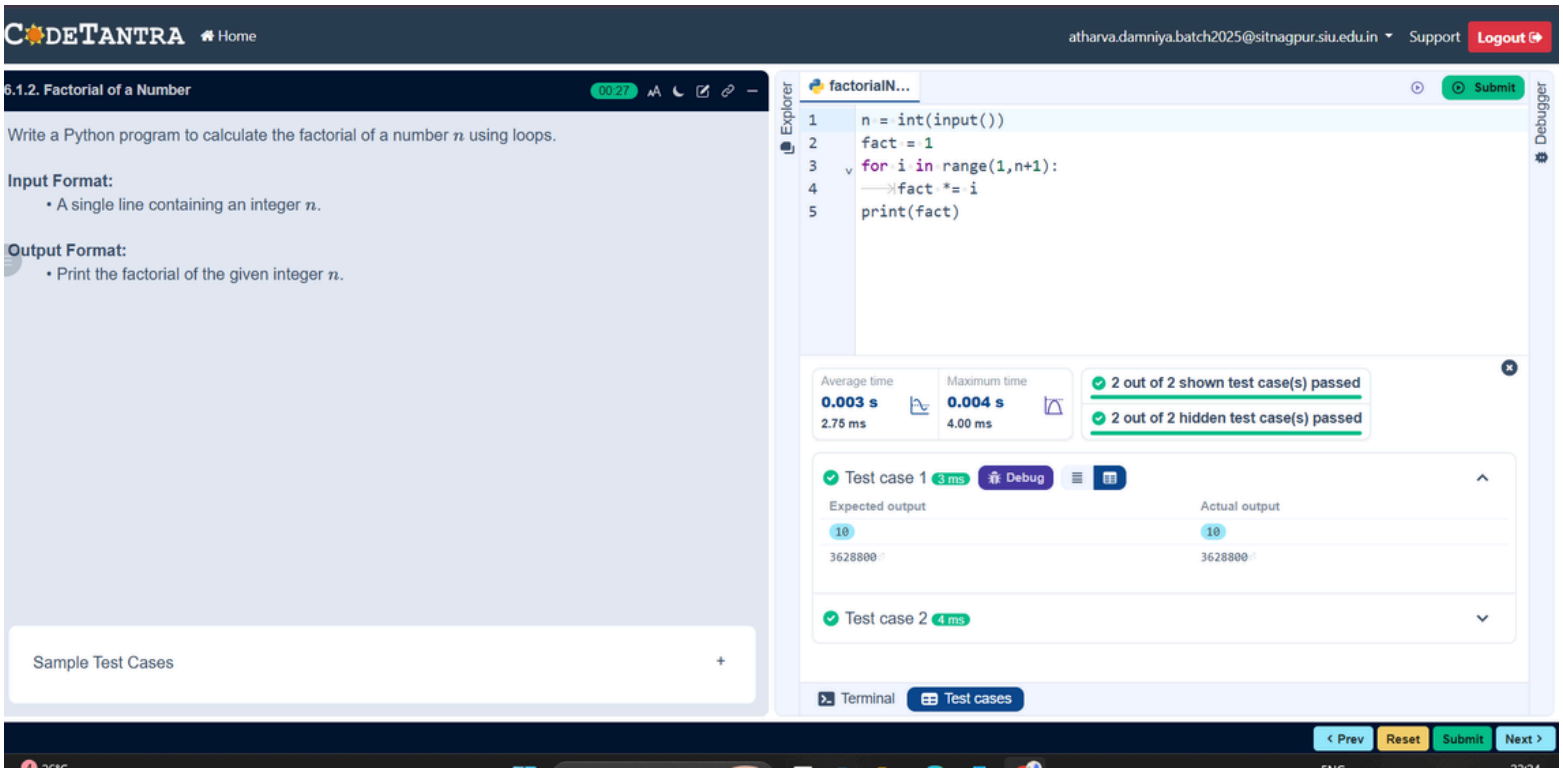
2. Pseudocode

1. **START**
2. **READ** a single integer from the user and **STORE** it in the variable n.
3. **INITIALIZE** a variable factorial to 1.
4. **FOR** each number i in the range from 1 to n (inclusive):
   - **MULTIPLY** factorial by i ($factorial = factorial * i$).
   - **STORE** the new value back into factorial.
5. **END FOR** loop once all numbers up to n have been multiplied.
6. **PRINT** the final value of factorial.
7. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 7.1.1: Inverted Star Pattern

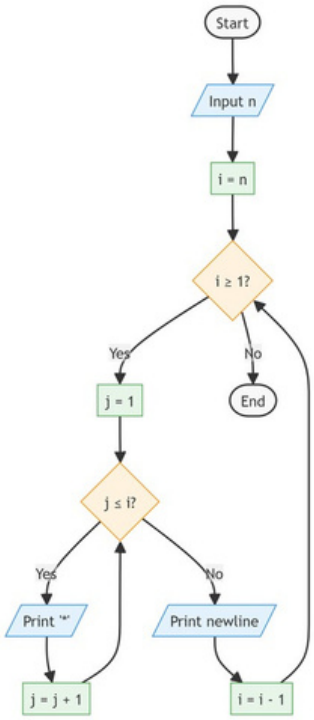
1. Aim

Todesignand implement a Python program that prints an inverted right-angled triangle star pattern. For a given integer n, the program uses nested loops to print n rows of stars, starting with n stars in the first row and decreasing by one star in each subsequent row until the last row has 1 star.

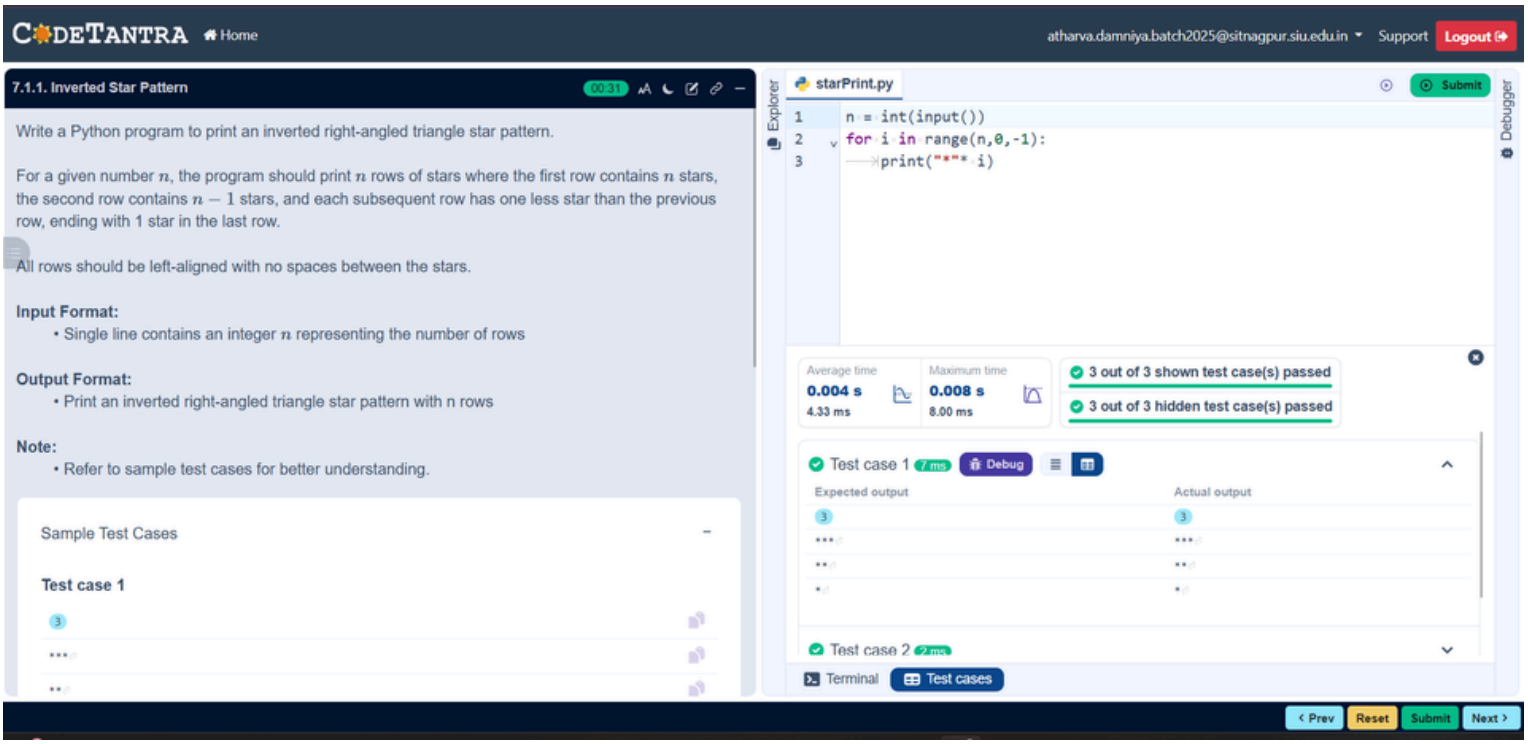
2. Pseudocode

1. **START**
2. **READ** a single integer from the user and **STORE** it in the variable n.
3. **FOR** each number i in the range starting from n down to 1 (decrementing by 1):
    - **FOR** each number j in the range from 0 to $i-1$:
        - **PRINT** a star **"*"** without a newline character at the end.
    - **ENDINNERFOR**
    - **PRINT** a newline character to move to the next row.
4. **END OUTERFOR**
5. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 7.1.2: Possible Combinations from Three Digits

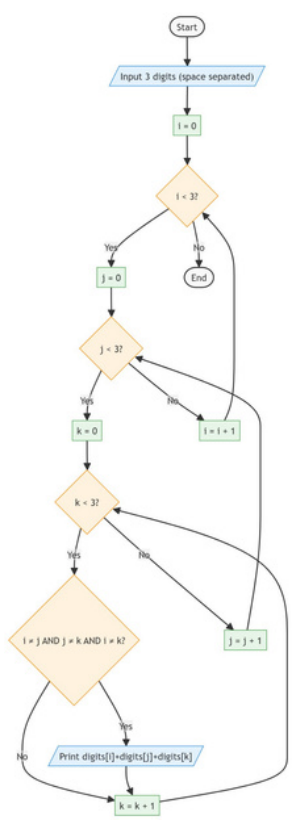
1. Aim

To design and implement a Python program that accepts three single-digit inputs (separated by spaces) and prints all six possible 3-digit combinations of these digits. Each digit should be used exactly once in each combination. The program maintains leading zeros if they are present.

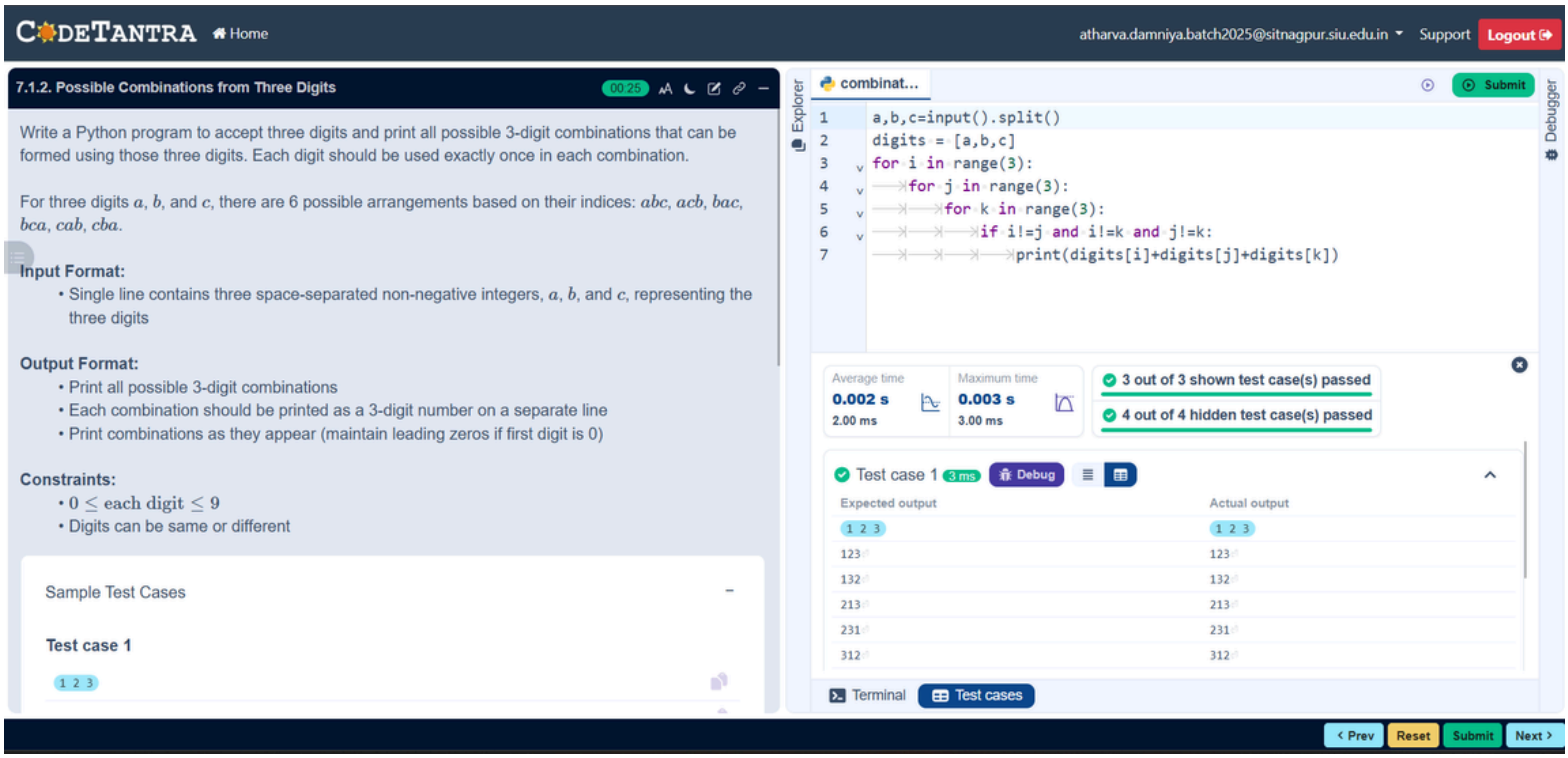
2. Pseudocode

1. **START**
2. **READ** a single line of input, split it by spaces, and **STORE** the three values into a list named digits.
3. **FOR** each index i from 0 to 2 (Outer Loop):
   - **FOR** each index j from 0 to 2 (Middle Loop):
     - **FOR** each index k from 0 to 2 (Inner Loop):
       - **CHECK IF** i is not equal to j **AND** j is not equal to k **AND** i is not equal to k.
       - **IF** the condition is True:
         - **PRINT** the concatenation of digits[i], digits[j], and digits[k].
4. **END LOOPS**
5. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 8.1.1: Multiplication of Two Square Matrices

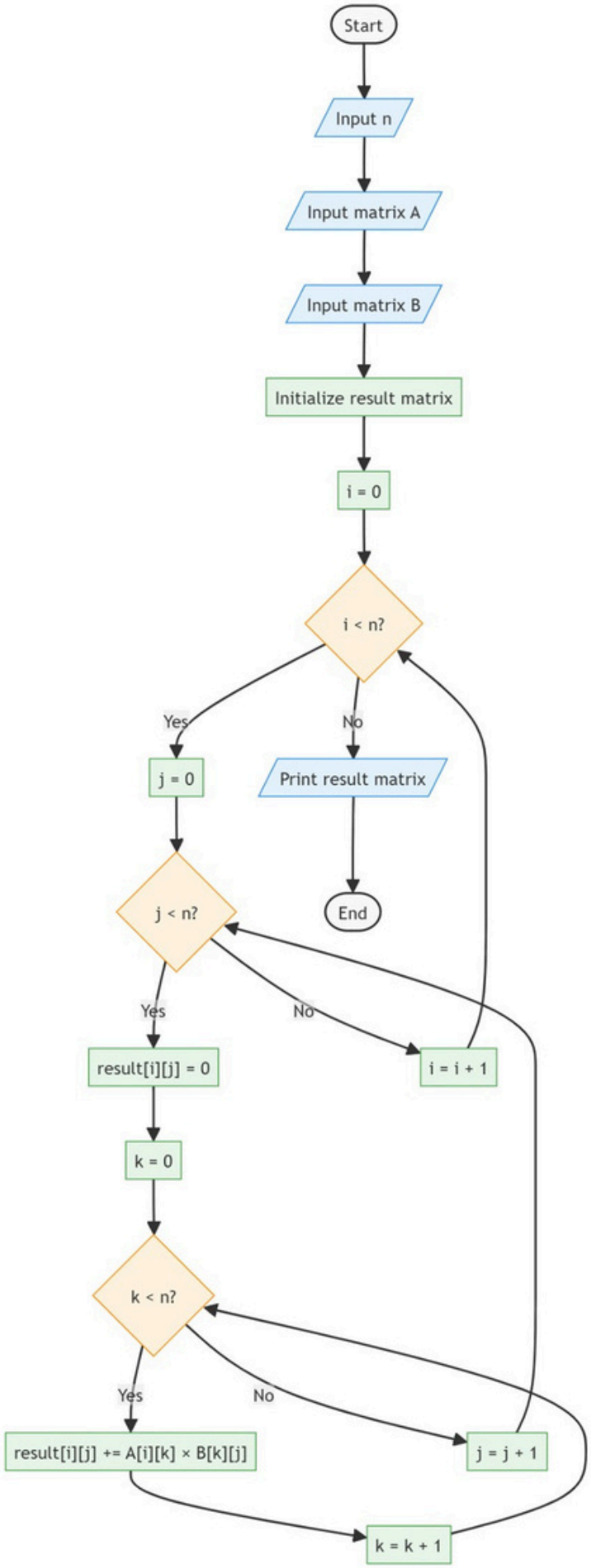
1. Aim

Todesign andimplement a Python program that performs the multiplication of two square matrices of dimension $n \times n$. The program takes the dimension and the elements of both matrices as input from the user and computes their product.

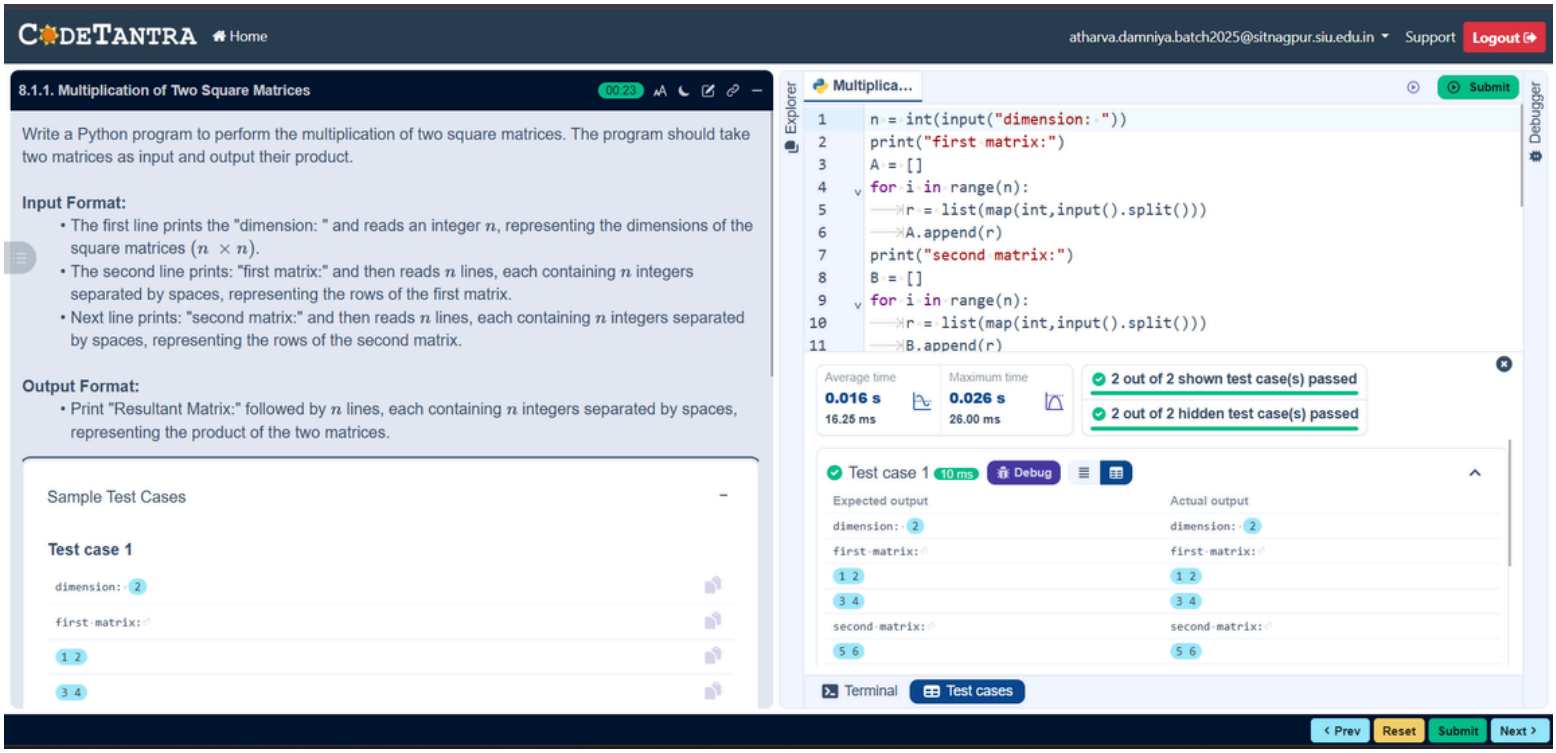
2. Pseudocode

1. **START**
2. **PRINT** "dimension: " and **READ** an integer from the user, storing it in the variable n.
3. **PRINT** "first matrix:" and **INITIALIZE** an empty list A.
4. **FOR** each row i from 0 to $n-1$:
   - **READ** space-separated integers from the user, convert them into a list, and append to A.
5. **PRINT** "second matrix:" and **INITIALIZE** an empty list B.
6. **FOR** each row i from 0 to $n-1$:
   - **READ** space-separated integers from the user, convert them into a list, and append to B.
7. **INITIALIZE** a result matrix of size $n \times n$ filled with zeros.
8. **FOR** i in range n (iterating through rows of A):
   - **FOR** j in range n (iterating through columns of B):
     - **FOR** k in range n (iterating through rows of B):
       - **ADD** the product of A[i][k] and B[k][j] to result[i][j].
9. **PRINT** "Resultant Matrix:".
10. **FOR** each row r in the result matrix:
    - **PRINT** the elements of r separated by spaces.
11. **END**

3. Flowchart


Start
Input n
Input matrix A
Input matrix B
Initialize result matrix
i = 0
i < n?
Yes
No
j = 0
Print result matrix
End
j < n?
Yes
No
result[i][j] = 0
i = i + 1
k = 0
k < n?
Yes
No
result[i][j] += A[i][k] × B[k][j]
j = j + 1
k = k + 1

4. Experiment Screenshot



Experiment 8.1.2: Prime Numbers in a Given Range

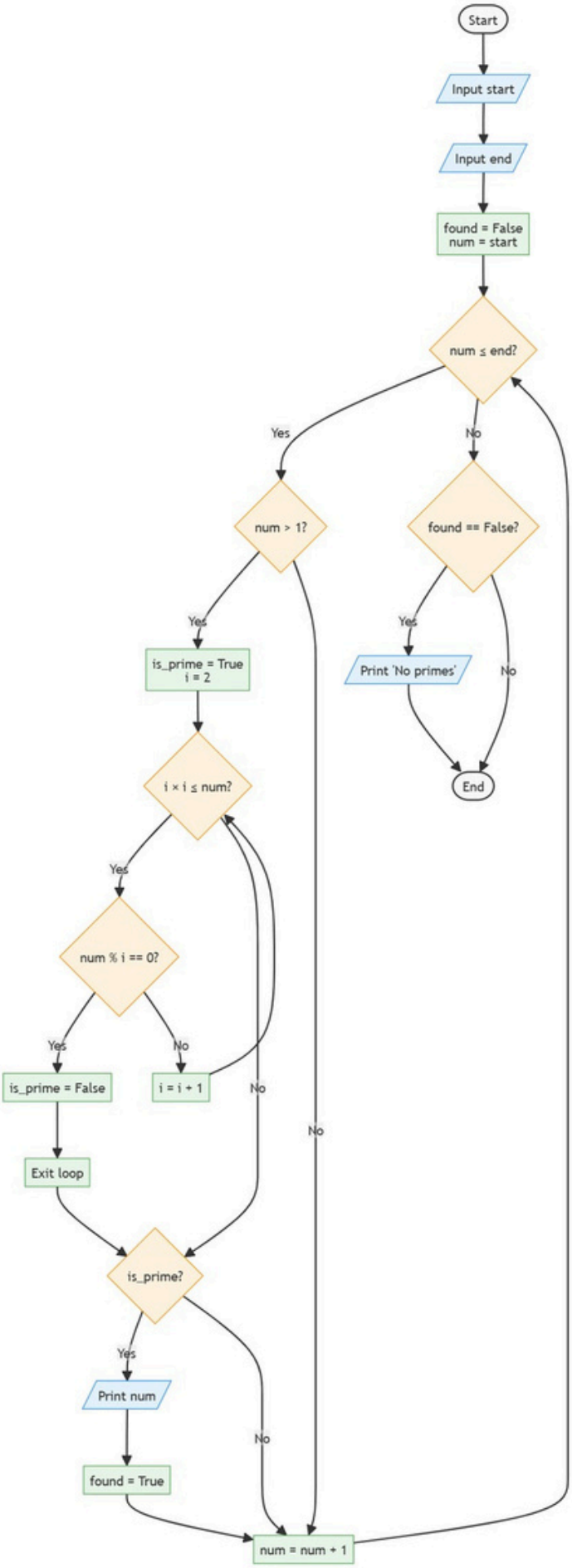
1. Aim

Todesign and implement a Python program that finds and prints all prime numbers within a specified range [start, end] (inclusive). The program determines the primality of each number and prints it. If no prime numbers exist within the given range, the program prints "No primes".

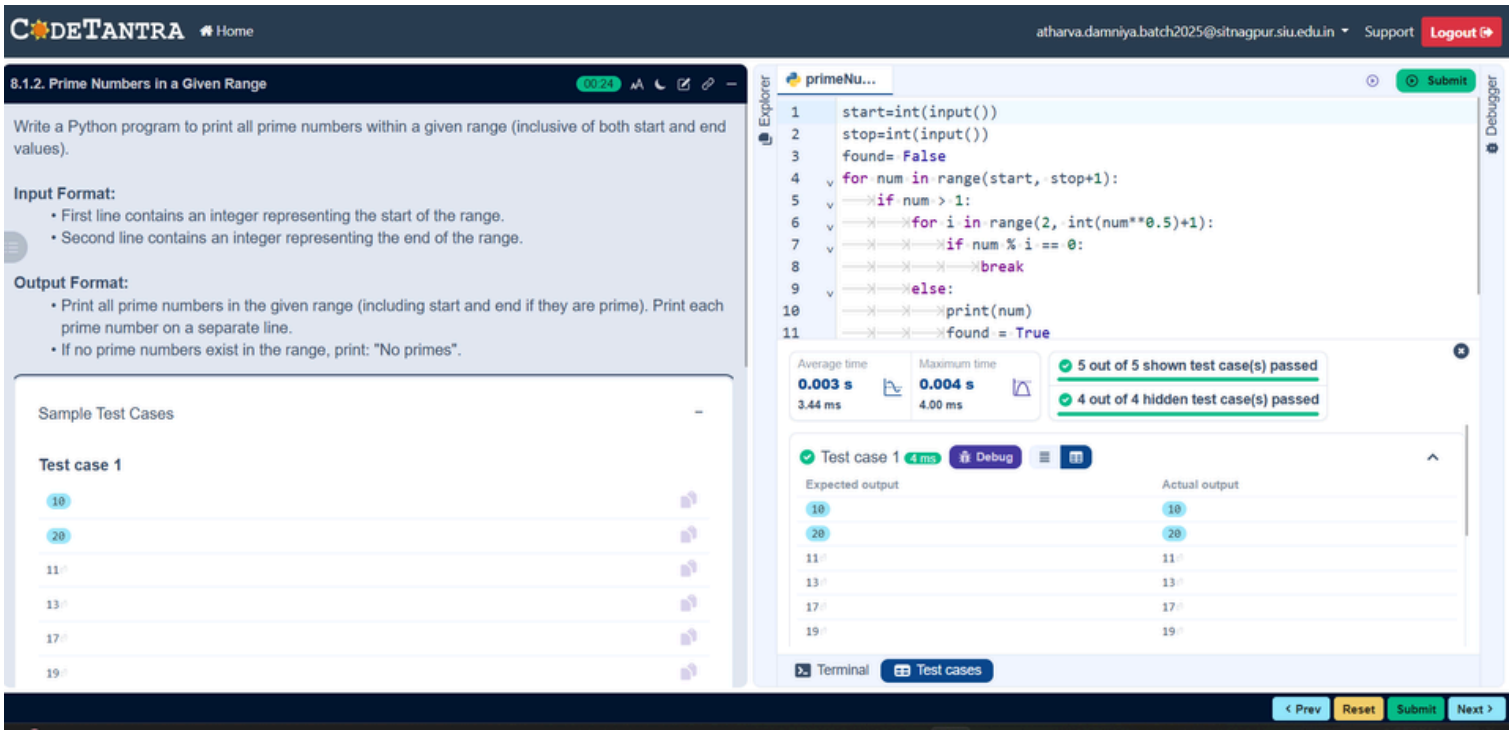
2. Pseudocode

1. **START**
2. **READ** an integer representing the start of the range and **STORE** it in start.
3. **READ** an integer representing the end of the range and **STORE** it in end.
4. **INITIALIZE** a boolean variable found to False.
5. **FOR** each number num in the range from start to end (inclusive):
   - **IF** num > 1:
     - **INITIALIZE** is_prime to True.
     - **FOR** each number i from 2 up to the integer square root of num ($int(num^{0.5}) + 1$):
       - **IF** num is divisible by i (num % i == 0):
         - **SET** is_prime to False.
         - **BREAK** the inner loop.
     - **IF** is_prime is True:
       - **PRINT** the prime number num.
       - **SET** found to True.
6. **IF** found is False:
   - **PRINT** "No primes".
7. **END**

3. Flowchart


Start
Input start
Input end
found = False
num = start
num ≤ end?
Yes
No
num > 1?
found == False?
Yes
No
is_prime = True
i = 2
Yes
Print 'No primes'
No
End
i × i ≤ num?
Yes
num % i == 0?
Yes
No
is_prime = False
i = i + 1
No
No
Exit loop
is_prime?
Yes
Print num
No
found = True
num = num + 1

4. Experiment Screenshot



Experiment 9.1.1: Count Vowels in a String - 1

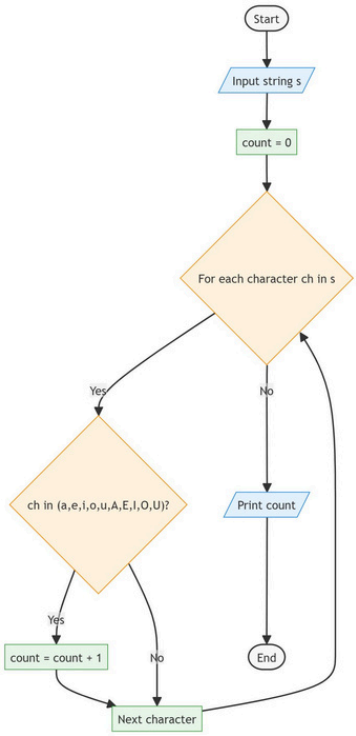
1. Aim

Todesign andimplement a Python program that takes a string as input and prints the total count of vowels present in that string. The program should consider both uppercase and lowercase vowels ('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U').

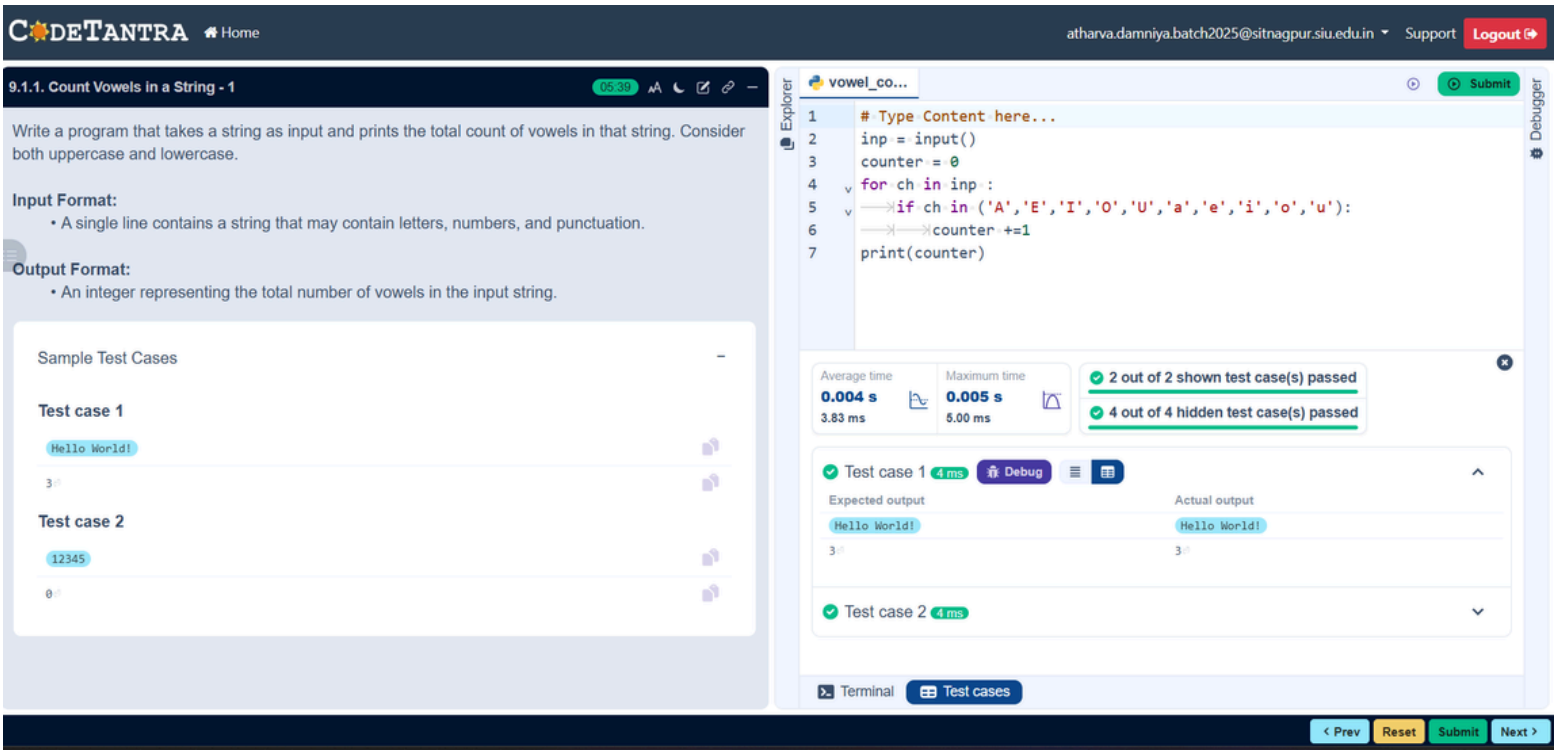
2. Pseudocode

1. **START**
2. **READ** the input string from the user and **STORE** it in the variable s.
3. **INITIALIZE** a variable count to 0 to keep track of the number of vowels.
4. **FOR** each character ch in the string s:
   - **CHECK IF** the character ch is present in the set of vowels "aeiouAEIOU".
   - **IF** the condition is True, **INCREMENT** count by 1.
5. **PRINT** the final value of count.
6. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 9.1.2: Palindrome or Not

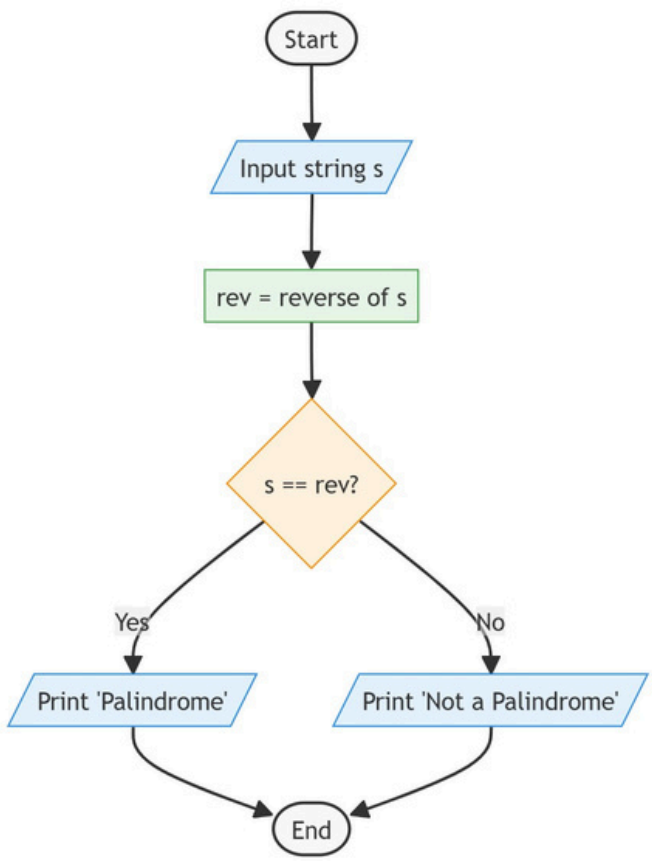
1. Aim

Todesignand implement a Python program that takes a string as input and checks whether the given string is a palindrome or not. A palindrome is a string that reads the same forwards and backwards.

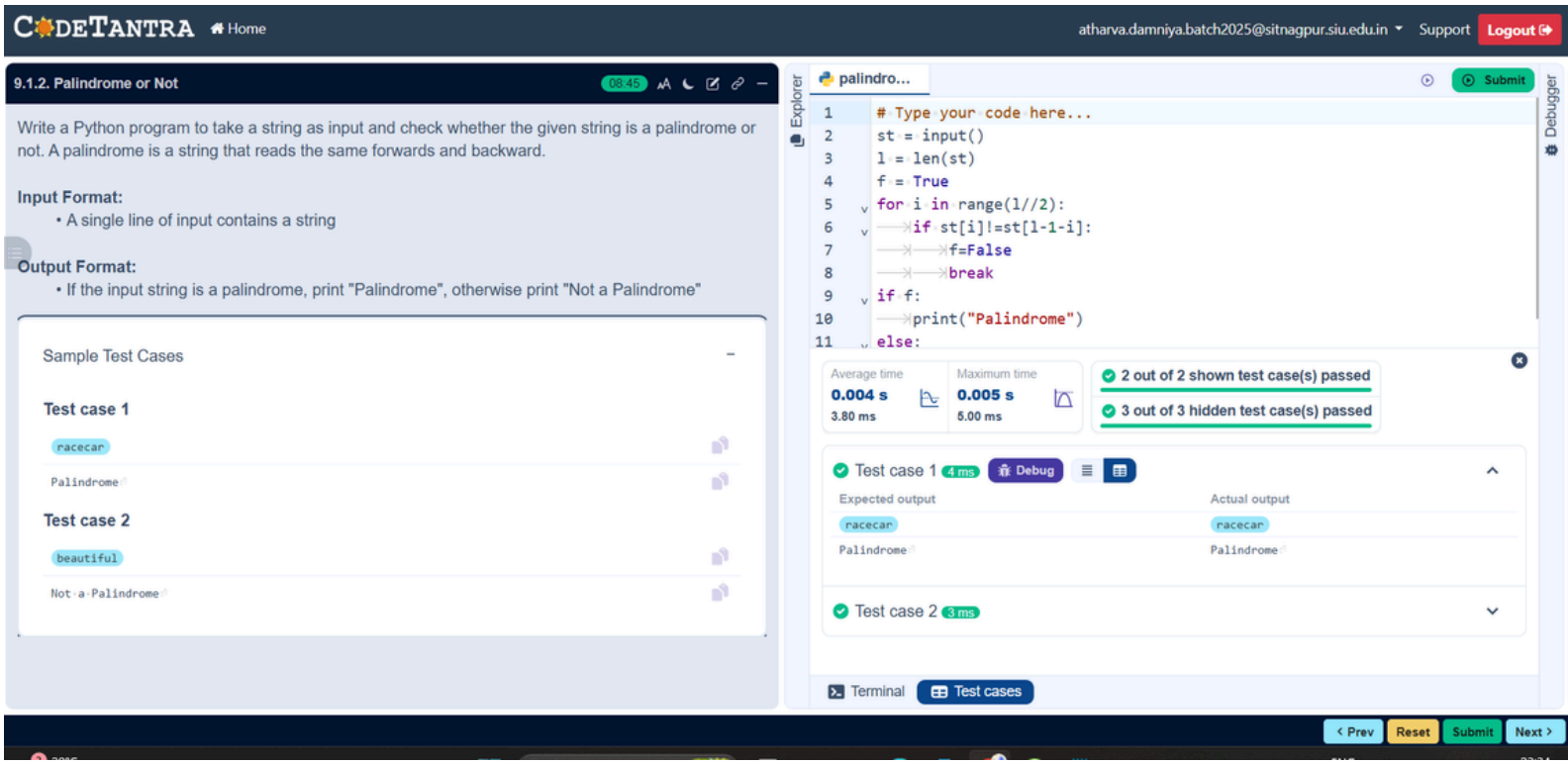
2. Pseudocode

1. **START**
2. **READ** the input string from the user and **STORE** it in the variable s.
3. **CREATE** a reversed version of the string s using slicing (s[::-1]) and **STORE** it in the variable rev.
4. **CHECK IF** the original string s is equal to the reversed string rev.
   - **IF** True, **PRINT** "Palindrome".
   - **ELSE**, **PRINT** "Not a Palindrome".
5. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 10.1.1: Remove Punctuations from a String

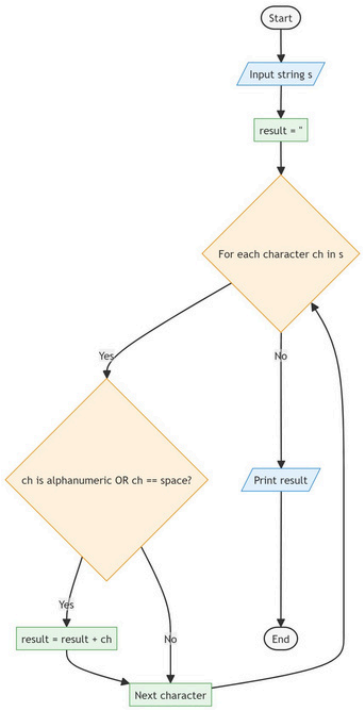
1. Aim

Todesignand implement a Python program that removes all punctuation marks from a given input string. Punctuation marks are defined as special characters that are not letters (uppercase or lowercase), digits, or spaces. The program should preserve alphanumeric characters and spaces while printing the final cleaned string.

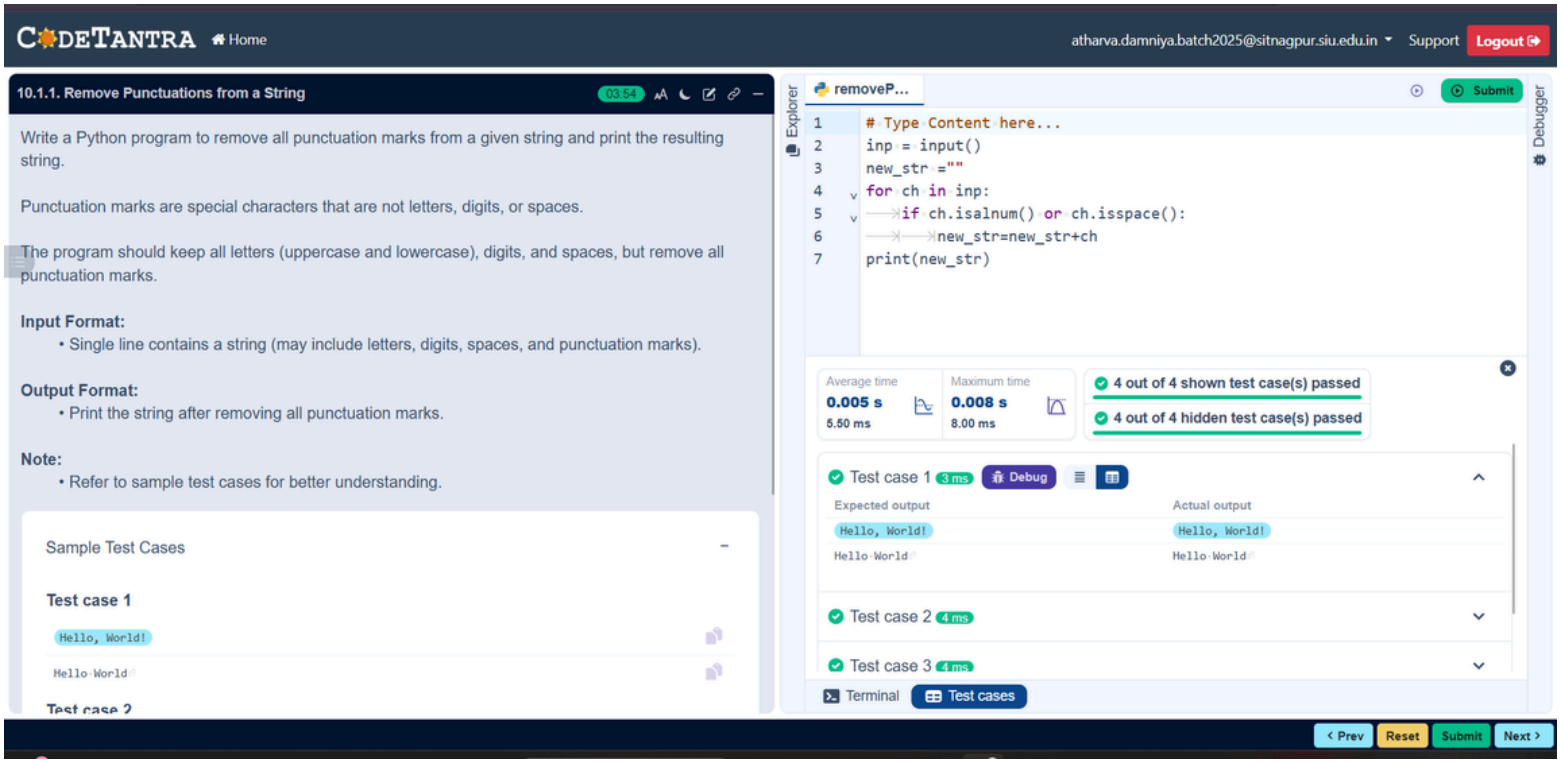
2. Pseudocode

1. **START**
2. **READ** the input string from the user and **STORE** it in the variable s.
3. **INITIALIZE** an empty string variable named result to store the cleaned text.
4. **FOR** each character ch in the input string s:
   - **CHECK IF** ch is an alphanumeric character (letter or digit) **OR** if ch is a space (" ").
   - **IF** the condition is True, **APPEND** the character ch to the result string.
5. **PRINT** the final value of the result string.
6. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 10.1.2: Reverse a String

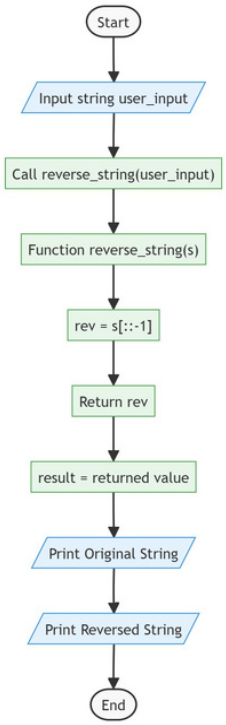
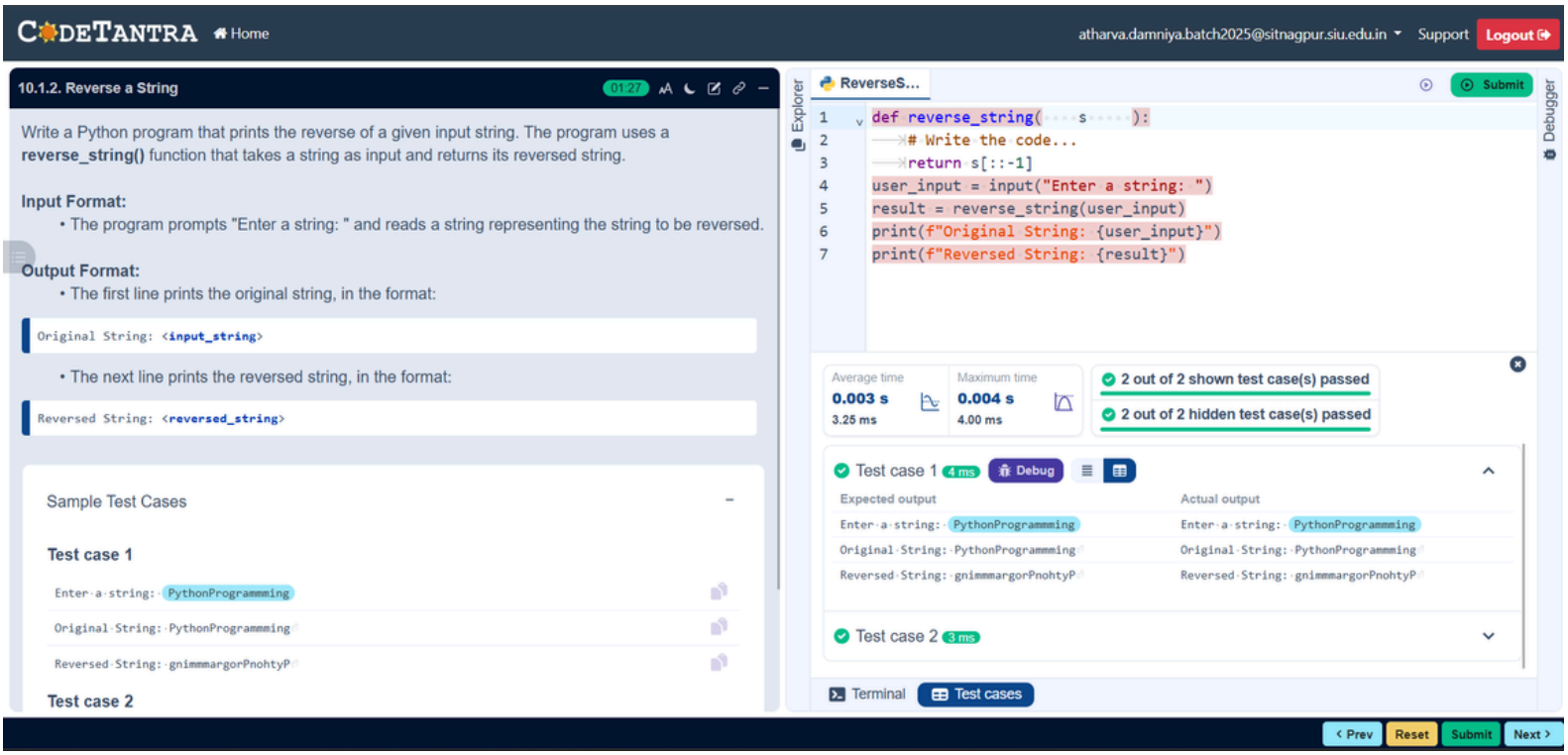
1. Aim

Todesign and implement a Python program that prints the reverse of a given input string. The program should use a user-defined function named reverse_string() that takes a string as input and returns its reversed version using string slicing.

2. Pseudocode

1. **START**
2. **DEFINE** a function reverse_string(s):
   - **RETURN** the reversed string using slicing: s[::-1].
3. **READ** the input string from the user with the prompt "Enter a string: " and **STORE** it in the variable user_input.
4. **CALL** the reverse_string() function with user_input as an argument and **STORE** the returned value in the variable result.
5. **PRINT** the original string in the format: "Original String: {user_input}".
6. **PRINT** the reversed string in the format: "Reversed String: {result}".
7. **END**

3. Flowchart



4. Experiment Screenshot